Informe Técnico

Preprocesamiento de imágenes del conjunto de datos Paddy Doctor para la clasificación de enfermedades del arroz

Curso: Visión por Computadora

Proyecto semestral

Universidad Nacional de San Antonio Abad del Cusco

Octubre de 2026

Resumen

El presente informe describe y justifica el flujo de preprocesamiento aplicado al conjunto de datos *Paddy Doctor*, compuesto por 10 407 imágenes de cultivos de arroz distribuidas en diez categorías (nueve enfermedades o plagas y una clase sana). El flujo implementado comprende: (i) la adquisición reproducible del conjunto mediante la API de Kaggle; (ii) un análisis exploratorio de la distribución de clases; (iii) la mejora del contraste mediante CLAHE sobre el canal de luminancia del espacio de color CIELAB; (iv) el redimensionamiento a 256 × 256 píxeles con interpolación por área; (v) un esquema de aumento de datos geométrico y fotométrico moderado; y (vi) la normalización con los estadísticos de ImageNet y su encapsulado en un *Dataset* y un *DataLoader* de PyTorch. Asimismo, se documenta la evaluación de una segmentación por umbralización HSV, que fue descartada del flujo final por eliminar tejido enfermo relevante. La verificación final confirma la generación de lotes de tensores de dimensión [32, 3, 256, 256] y la compatibilidad con una red EfficientNet-B0 preentrenada.

Palabras clave: preprocesamiento de imágenes, CLAHE, espacio de color LAB, aumento de datos, enfermedades del arroz, aprendizaje profundo, PyTorch.

1. Introducción

El arroz es un cultivo básico para gran parte de la población mundial, y su producción se ve amenazada por enfermedades y plagas que, detectadas tarde, generan pérdidas considerables. La identificación visual por parte de especialistas es costosa y no siempre está disponible, lo que motiva el uso de modelos de visión por computadora capaces de clasificar automáticamente imágenes de plantas afectadas.

El rendimiento de estos modelos depende en gran medida de la calidad y la consistencia de los datos de entrada. Las imágenes tomadas en campo presentan variaciones de iluminación, sombras, fondos complejos y diferentes resoluciones, factores que introducen ruido no relacionado con la enfermedad. El preprocesamiento busca reducir dicha variabilidad espuria, resaltar la información visual discriminativa y entregar al modelo tensores con un formato homogéneo.

Objetivo general. Documentar de forma completa el preprocesamiento aplicado a las imágenes del conjunto Paddy Doctor, explicando cada decisión técnica y su justificación.

Objetivos específicos.

- Describir la adquisición y la estructura del conjunto de datos.
- Caracterizar la distribución de clases y sus implicancias.
- Explicar las transformaciones de mejora de imagen, redimensionamiento, aumento y normalización.
- Exponer las limitaciones del flujo actual y proponer mejoras.

El trabajo se desarrolló en Python, empleando OpenCV, Albumentations, PyTorch y la biblioteca *timm*, en un cuaderno de Jupyter diseñado para Google Colab.

2. Descripción del conjunto de datos

2.1 Origen y adquisición

El conjunto proviene de la competencia *Paddy Doctor: Paddy Disease Classification* de Kaggle. Su descarga se automatizó en el cuaderno mediante la herramienta de línea de comandos de Kaggle, que requiere un archivo de credenciales (`kaggle.json`) almacenado en `~/.kaggle` con permisos restrictivos (`chmod 600`) para evitar su lectura por otros usuarios del sistema. El código verifica si las credenciales y los datos ya existen antes de repetir la operación, y captura el error HTTP 403 que se produce cuando el usuario no ha aceptado las reglas de la competencia, mostrando un mensaje orientativo. Esta lógica garantiza que el cuaderno sea idempotente y reproducible: puede ejecutarse varias veces sin descargar de nuevo el archivo comprimido (`paddy-disease-classification.zip`), que se extrae en el directorio `./paddy_data`.

2.2 Estructura

Las imágenes de entrenamiento se organizan en subcarpetas por clase (`train_images/<clase>/<imagen>.jpg`) y un archivo `train.csv` asocia cada imagen con sus metadatos. Las primeras filas muestran las columnas `image_id`, `label`, `variety` (variedad de arroz, por ejemplo ADT45) y `age` (edad del cultivo en días). En este trabajo se emplean únicamente `image_id` y `label`.

En total se registran **10 407 imágenes** y **10 clases**: *normal* (sana), *blast*, *hispa*, *dead_heart*, *tungro*, *brown_spot*, *downy_mildew*, *bacterial_leaf_blight*, *bacterial_leaf_streak* y *bacterial_panicle_blight*.

3. Entorno de ejecución y reproducibilidad

Se fijó un tamaño de imagen de 256 × 256 píxeles, un tamaño de lote de 32 y la selección automática del dispositivo (GPU CUDA si está disponible; en caso contrario, CPU). La verificación reportada en el cuaderno se ejecutó en CPU, aunque el entorno recomendado es Google Colab con GPU T4.

Para favorecer la reproducibilidad se implementó la función `seed_everything(seed=42)`, que fija la semilla de los generadores aleatorios de Python, NumPy y PyTorch (CPU y GPU), la variable de entorno `PYTHONHASHSEED` y activa el modo determinista de cuDNN. La razón es que el aumento de datos, la inicialización de capas nuevas y el orden de los lotes son procesos estocásticos: sin una semilla fija, los resultados de dos ejecuciones no serían comparables. Debe señalarse que el determinismo completo en entornos con varios procesos de carga de datos exige además inicializar la semilla de cada *worker*, aspecto no implementado en esta versión.

4. Análisis exploratorio: distribución de clases

Antes de transformar los datos se examinó la frecuencia de cada categoría, pues una distribución desbalanceada condiciona la elección de métricas, estrategias de muestreo y funciones de pérdida. La Figura 1 y la Tabla 1 resumen el resultado.

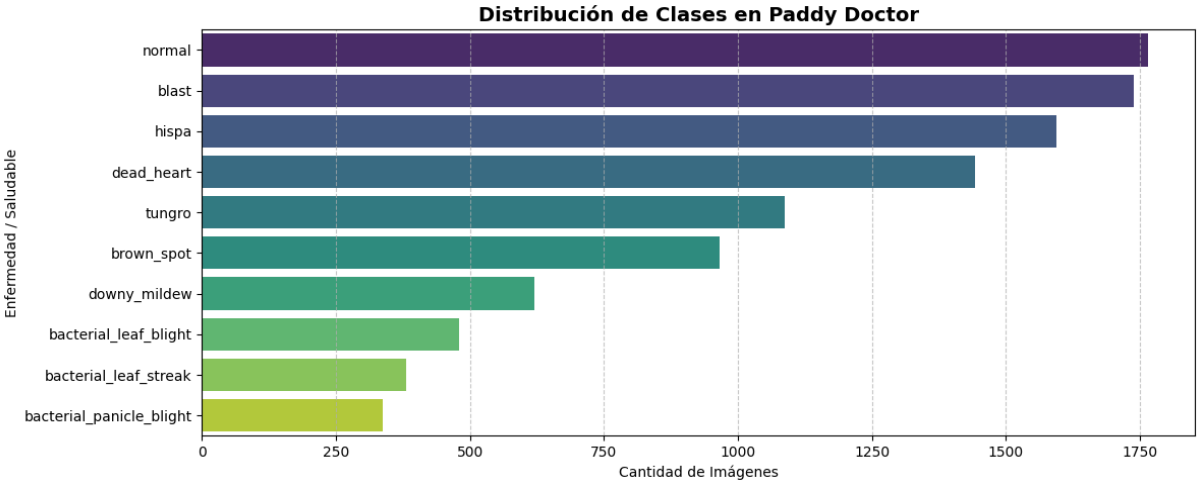


Figura 1. Distribución de imágenes por clase en el conjunto Paddy Doctor (salida del cuaderno).

Tabla 1. Número de imágenes por clase (valores leídos del conjunto; su suma coincide con el total registrado de 10 407).

Clase	Imágenes	Proporción
normal	1 764	17.0 %
blast	1 738	16.7 %
hispa	1 594	15.3 %
dead_heart	1 442	13.9 %

Clase	Imágenes	Proporción
tungro	1 088	10.5 %
brown_spot	965	9.3 %
downy_mildew	620	6.0 %
bacterial_leaf_blight	479	4.6 %
bacterial_leaf_streak	380	3.7 %
bacterial_panicle_blight	337	3.2 %
Total	**10 407**	**100 %**

Se observa un desbalance moderado: la clase mayoritaria (*normal*, 1 764 imágenes) supera en aproximadamente 5,2 veces a la minoritaria (*bacterial_panicle_blight*, 337 imágenes). Este hallazgo justifica el uso de técnicas de aumento de datos y, en la fase de entrenamiento, de métricas robustas al desbalance (F1 macro, matriz de confusión) y de una partición estratificada.

5. Preprocesamiento de las imágenes

El flujo unificado se implementó en la función `full_preprocessing_pipeline` y se compone de tres etapas secuenciales, resumidas en la Tabla 2.

Etapa	Operación	Parámetros	Propósito
1	Lectura con OpenCV (BGR)	Validación de lectura	Detectar archivos corruptos
2	CLAHE sobre canal L (LAB)	clipLimit = 3,0; mosaico 8 × 8	Normalizar iluminación y realzar textura
3	Redimensionamiento	256 × 256; INTER_AREA	Tamaño homogéneo de entrada
4*	Aumento de datos	Ver Sección 7	Regularización (solo entrenamiento)
5*	Normalización + tensor	Media/desv. de ImageNet	Compatibilidad con el modelo preentrenado

Tabla 2. Etapas del flujo de preprocesamiento. (*) Etapas aplicadas en el objeto Dataset.

5.1 Lectura y validación

Cada imagen se lee con `cv2.imread`, que devuelve una matriz en orden de canales BGR. Si la lectura falla (archivo ausente o dañado), `imread` devuelve `None` sin lanzar excepción; por ello la función verifica este caso y emite un error explícito con la ruta afectada. De este modo los fallos se detectan de inmediato y no se propagan silenciosamente como tensores inválidos.

5.2 Mejora de contraste con CLAHE en el espacio LAB

Las fotografías de campo presentan zonas sobreexpuestas, sombras y bajo contraste entre lesión y tejido sano. Para atenuar este problema se aplicó la ecualización adaptativa del histograma con limitación de contraste (CLAHE, *Contrast Limited Adaptive Histogram Equalization*). A diferencia de la ecualización global, CLAHE divide la imagen en un mosaico de regiones (aquí 8 × 8) y ecualiza el histograma de cada una, de modo que el realce se adapta a las condiciones locales de iluminación. El parámetro *clipLimit* (3,0) recorta los picos del histograma antes de ecualizar, limitando la amplificación del ruido en regiones homogéneas.

La decisión clave consiste en **no aplicar CLAHE directamente sobre los canales RGB**, sino convertir la imagen al espacio CIELAB y operar solo sobre el canal *L* (luminancia). En RGB, ecualizar cada canal por separado altera sus proporciones relativas y distorsiona el color, y el color es precisamente un indicio diagnóstico fundamental (manchas pardas, amarillamiento, clorosis). En LAB, la luminancia está separada de la información cromática (canales *a* y *b*), de manera que el contraste mejora sin modificar la tonalidad. Tras el realce, los canales se recombinan y se convierten a RGB.

5.3 Redimensionamiento

Las redes convolucionales requieren entradas de tamaño fijo para formar lotes. Se redimensionó cada imagen a 256 × 256 píxeles, resolución que equilibra el detalle de las lesiones con el costo computacional y la memoria, y que deja margen para recortes o desplazamientos posteriores. Se eligió la interpolación `INTER_AREA`, recomendada para reducir el tamaño de imágenes, ya que promedia el área de píxeles de origen y evita el *aliasing* (patrones de moiré) que producen otros métodos al submuestrear. Cabe notar que las imágenes mostradas tienen orientación vertical, por lo que la conversión a formato cuadrado no conserva la relación de aspecto y deforma ligeramente las proporciones; es una simplificación aceptada por su sencillez.

El orden de las operaciones también es deliberado: CLAHE se aplica *antes* del redimensionamiento, sobre la resolución original, para aprovechar el máximo detalle local al calcular los histogramas.

5.4 Segmentación HSV evaluada y descartada

Como alternativa, se implementó la función `apply_hsv_mask`, que convierte la imagen al espacio HSV y conserva solo los píxeles con matiz entre 25 y 90 (verdes y amarillo-verdosos), saturación y valor mínimos de 30. La hipótesis era eliminar fondos no vegetales (suelo, agua, sombras) y concentrar la atención del modelo en la planta.

La comparación visual (Figura 2) mostró un efecto adverso: la máscara elimina también las lesiones, cuyo color pardo, marrón o rosado queda fuera del rango verde (se observan como zonas negras sobre las hojas, en especial en *bacterial_leaf_streak* y *bacterial_panicle_blight*). Dado que dichas lesiones constituyen la señal discriminativa de la enfermedad, esta segmentación destruiría información útil. Por este motivo la función se conservó solo con fines de análisis comparativo y **no se incluyó en el flujo final**. En contraste, la columna central evidencia que CLAHE realza la textura y el contraste de las hojas manteniendo los colores naturales.

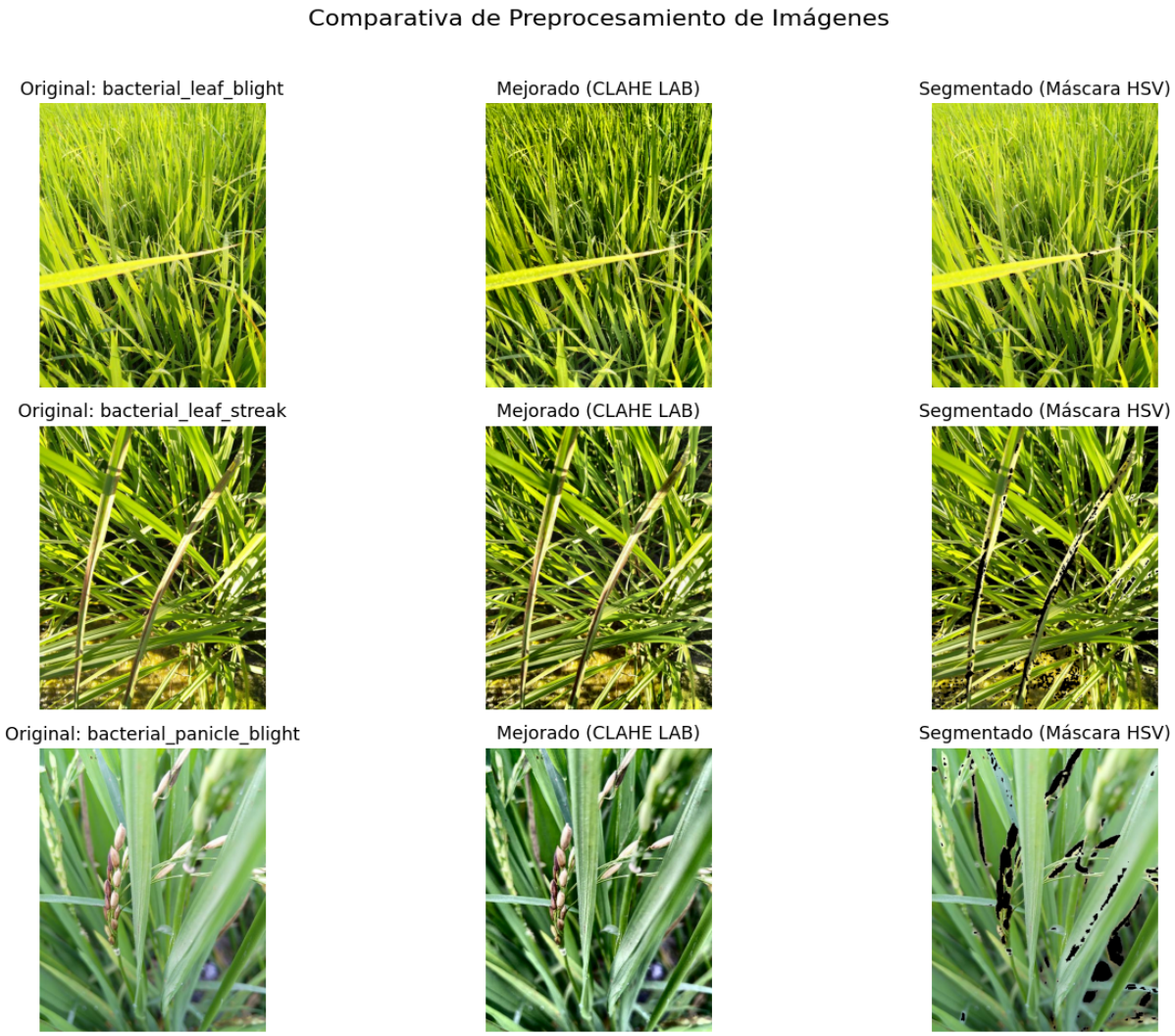


Figura 2. Comparación entre la imagen original, la imagen mejorada con CLAHE-LAB y la segmentación por máscara HSV para tres clases (salida del cuaderno).

6. Encapsulado en un Dataset y DataLoader de PyTorch

Para integrar el preprocesamiento con el entrenamiento se definió la clase `PaddyDataset`, que hereda de `torch.utils.data.Dataset`. Sus características principales son:

- **Carga perezosa (*lazy loading*):** las imágenes se leen y procesan bajo demanda en `__getitem__`, evitando cargar 10 407 imágenes en memoria simultáneamente.
- **Mapeo de clases:** las etiquetas textuales se ordenan alfabéticamente y se convierten a índices enteros (0–9), formato requerido por las funciones de pérdida de clasificación. El orden alfabético garantiza un mapeo determinista y reproducible.
- **Rutas por clase:** la ruta se construye a partir de la etiqueta y el identificador de la imagen, según la estructura de carpetas descrita en la Sección 2.
- **Transformaciones opcionales:** el parámetro `transform` permite aplicar aumento de datos solo al conjunto de entrenamiento.

El `DataLoader` se configuró con lotes de 32 imágenes, mezcla aleatoria (`shuffle=True`) para evitar que el modelo aprenda el orden de los datos, dos procesos de carga en paralelo

(`num_workers=2`) para solapar la lectura de disco con el cómputo, y `pin_memory=True` para acelerar la transferencia a la GPU (sin efecto cuando se ejecuta en CPU, como advirtió el propio sistema).

7. Aumento de datos

El aumento de datos genera variantes plausibles de cada imagen durante el entrenamiento, ampliando artificialmente la diversidad del conjunto, reduciendo el sobreajuste y mitigando parcialmente el desbalance de clases. Se empleó la biblioteca Albumentations con las transformaciones de la Tabla 3.

Transformación	Parámetros	Justificación
HorizontalFlip	p = 0,5	Las hojas no tienen lateralidad canónica
VerticalFlip	p = 0,5	Las fotografías de campo no tienen orientación fija
RandomRotate90	p = 0,5	Invarianza a rotaciones de 90°
ColorJitter	brillo 0,1; contraste 0,2; saturación 0,2; p = 0,5	Simula variaciones de iluminación y cámara
ShiftScaleRotate	desplazamiento 0,05; escala 0,1; rotación 30°; p = 0,5	Simula cambios de encuadre, distancia y ángulo

Tabla 3. Transformaciones de aumento de datos aplicadas al conjunto de entrenamiento.

Las magnitudes de las transformaciones fotométricas son deliberadamente moderadas y **no se modificó el matiz** (*hue*), porque el color de las lesiones es un rasgo diagnóstico; alterarlo podría generar ejemplos que contradigan su etiqueta. Las transformaciones geométricas son apropiadas porque la clase de una imagen no depende de su orientación. Cada transformación se aplica de manera probabilística, por lo que cada época presenta al modelo versiones distintas de una misma imagen.

8. Normalización y conversión a tensor

Como última etapa se aplicó `A.Normalize` con la media (0,485; 0,456; 0,406) y la desviación estándar (0,229; 0,224; 0,225) de ImageNet, seguida de `ToTensorV2`, que reordena la matriz de (alto, ancho, canal) a (canal, alto, ancho) y la convierte en tensor de PyTorch. Se emplearon los estadísticos de ImageNet porque el modelo utilizado (EfficientNet-B0) fue preentrenado con ellos: para aprovechar adecuadamente el aprendizaje por transferencia, la distribución de los datos de entrada debe ser similar a la del preentrenamiento. Además, centrar y escalar las entradas mejora la estabilidad numérica y acelera la convergencia del descenso por gradiente.

9. Verificación del flujo

Se realizaron dos comprobaciones para validar el flujo de extremo a extremo:

- Al extraer un lote del *DataLoader*, se obtuvo un tensor de imágenes de dimensión `[32, 3, 256, 256]` y un tensor de etiquetas de dimensión `[32]`, lo que confirma el tamaño, el orden de los canales y el tamaño de lote esperados.

- Se instanció un clasificador basado en EfficientNet-B0 (biblioteca *timm*, pesos preentrenados, 10 clases de salida) y se ejecutó una pasada hacia adelante sin cálculo de gradientes, obteniéndose una salida de dimensión `[32, 10]` (*logits* por clase), coherente con el número de categorías.

Estas pruebas demuestran que los datos preprocesados son compatibles con el modelo y que el flujo está listo para la fase de entrenamiento.

10. Limitaciones y trabajo futuro

El flujo cumple su propósito, pero se identifican aspectos susceptibles de mejora:

- **Ausencia de partición de datos:** el cuaderno no divide el conjunto en entrenamiento y validación. Se recomienda una partición estratificada (por ejemplo 80/20) para conservar las proporciones de clase, y aplicar el aumento de datos únicamente al subconjunto de entrenamiento.
- **Desbalance de clases:** no se aplicó ninguna corrección explícita. Podrían emplearse pesos de clase, muestreo ponderado o pérdida focal.
- **Costo computacional:** CLAHE y el redimensionamiento se recalculan en cada acceso a una imagen y en cada época. Al ser transformaciones deterministas, podrían precalcularse una sola vez y almacenarse en disco, reduciendo el tiempo de entrenamiento.
- **Relación de aspecto:** podría conservarse mediante relleno (*padding*) o recorte central en lugar de la deformación a formato cuadrado.
- **Validación empírica de CLAHE:** su beneficio se justificó con inspección visual; un estudio de ablación que compare el desempeño del modelo con y sin CLAHE permitiría cuantificar su aporte.
- **Reproducibilidad total:** inicializar la semilla de los procesos de carga de datos.
- **Compatibilidad de la biblioteca:** el sistema advirtió que `ShiftScaleRotate` es un caso particular de `Affine` y será retirado; conviene migrar a esta última.
- **Seguridad de credenciales:** el archivo `kaggle.json` contiene una clave privada y no debe compartirse ni incluirse en repositorios.

11. Conclusiones

Se diseñó y verificó un flujo de preprocesamiento reproducible para el conjunto Paddy Doctor. El análisis exploratorio reveló un desbalance moderado entre clases (relación aproximada de 5,2 a 1). El realce de contraste mediante CLAHE sobre el canal de luminancia del espacio LAB mejora la visibilidad de la textura foliar sin alterar la información cromática, mientras que la segmentación HSV fue descartada tras comprobarse que elimina las lesiones que caracterizan las enfermedades. El redimensionamiento con interpolación por área, el aumento de datos moderado y la normalización con los estadísticos de ImageNet completan una canalización eficiente y compatible con el aprendizaje por transferencia. Las limitaciones señaladas constituyen la hoja de ruta para la fase de entrenamiento y evaluación del clasificador.

Referencias

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and flexible image augmentations. *Information*, 11(2), 125.

Kaggle. (s. f.). *Paddy Doctor: Paddy Disease Classification* [Competencia]. https://www.kaggle.com/c/paddy-disease-classification

Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

Pizer, S. M., et al. (1987). Adaptive histogram equalization and its variations. *Computer Vision, Graphics, and Image Processing*, 39(3), 355–368.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning*, 6105–6114.

Wightman, R. (2019). PyTorch Image Models (timm). GitHub. https://github.com/huggingface/pytorch-image-models